

VITyarthi – Build Your Own Project

PROJECT REPORT

To-Do List Management System

Submitted By

Name: Labhanshi Ahire

Registration / Roll No.: 26MIB10048

Branch: Integrated M.Tech Bioinformatics and AI

Year / Semester: 2026-27 (fall semester)

Subject: CSE- Introduction to problem solving in python

Faculty: Dr. Sasmita Padhy

Academic Year: 2026-27

Submitted as part of the VITyarthi Build Your Own Project evaluation.

Table of Contents

1. Introduction	
2. Problem Statement	
3. Objectives	
4. Functional Requirements	
5. Non-Functional Requirements	
6. System Architecture	
7. Workflow Diagram	
8. Use Case Diagram	
9. Component Diagram	
10. Sequence Diagram	
11. Storage Design	
12. Design Decisions and Rationale	
13. Implementation Details	
14. Screenshots / Results	8–13
15. Testing Approach	14
16. Challenges Faced	14
17. Learnings and Key Takeaways	14
18. Future Enhancements	14–15
19. Conclusion	15
20. References	15

1. Introduction

The To-Do List Management System is a simple Python program made to help students manage their daily academic and personal tasks. A user can add a task, view tasks, edit a task, mark a task as completed, delete a task and view a simple report.

The project uses basic Python concepts such as variables, input and output, if/else statements, loops, functions, lists, dictionaries and file handling. The tasks are stored in a CSV file so that they can be loaded again when the program starts.

The project is divided into small Python files. This makes the program easier for a first-year student to understand, test and maintain.

2. Problem Statement

Students often have many academic and personal tasks. If these tasks are written in different places or remembered only mentally, it can become difficult to track pending work and important tasks.

The proposed solution is a simple command-line To-Do List Management System. It allows the user to record tasks, give them a category and priority, track their status and view task information.

3. Objectives

- Develop a simple task management application using Python.
- Allow users to add, view, edit, complete and delete tasks.
- Organize tasks using category and priority.
- Store task information permanently in a CSV file.
- Validate user input and handle common errors.
- Generate a simple report of task statistics.
- Practice modular programming by separating the project into files.

4. Functional Requirements

Module	Functions
Task Management	Add, view, edit, complete and delete tasks.
Validation	Check task numbers, priority values and empty task names.
Data Storage	Read and write task information using a CSV file.
Reporting	Show total, completed, pending, priority and category information.

The user interacts with the system through a menu. Each menu option performs one task and gives a clear message as output.

5. Non-Functional Requirements

Usability: The menu and messages are simple so that a beginner can use the application.

Reliability: Tasks are saved to a CSV file so that they remain available after the program is closed.

Error Handling: Invalid task numbers, invalid priorities and non-numeric input are handled.

Maintainability: The code is divided into separate files, with each file having a clear purpose.

Resource Efficiency: The project uses a small CSV file and Python standard library modules, so no external package is required.

6. System Architecture

The following diagram shows how the main parts of the project work together.

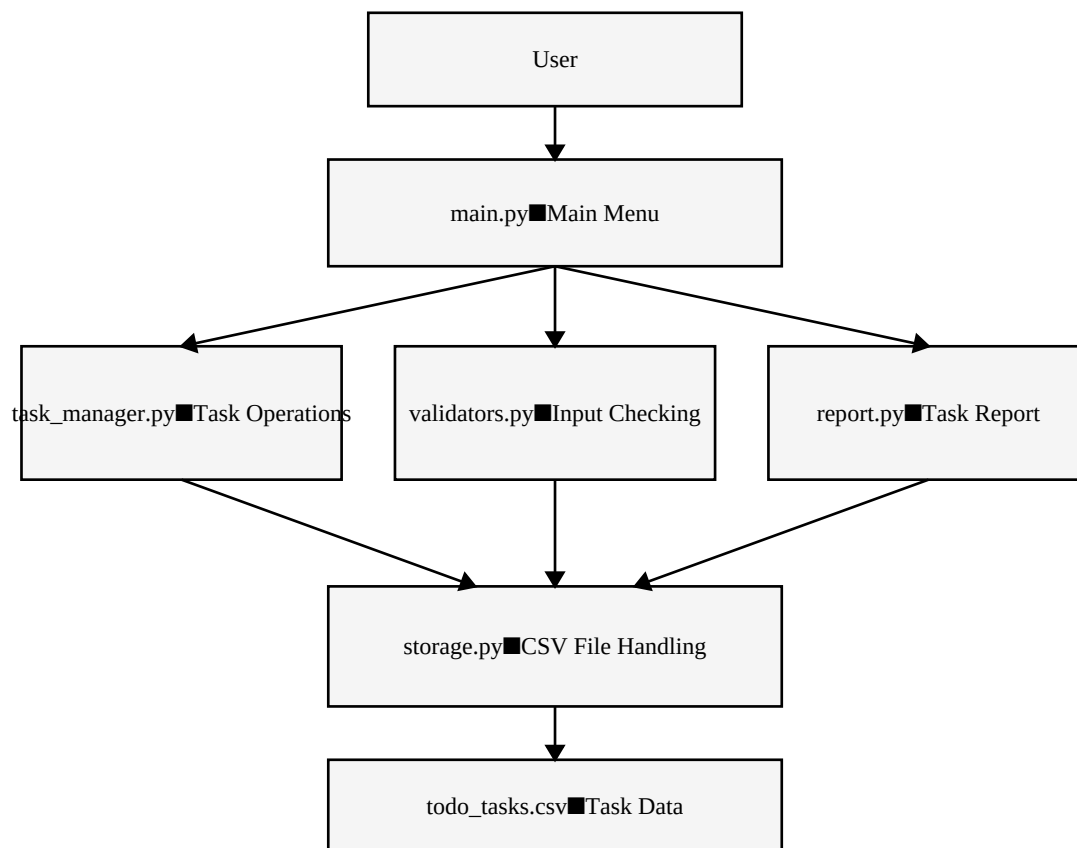


Figure 1: System architecture of the To-Do List Management System.

7. Workflow Diagram

The workflow shows the normal interaction between the user and the program.

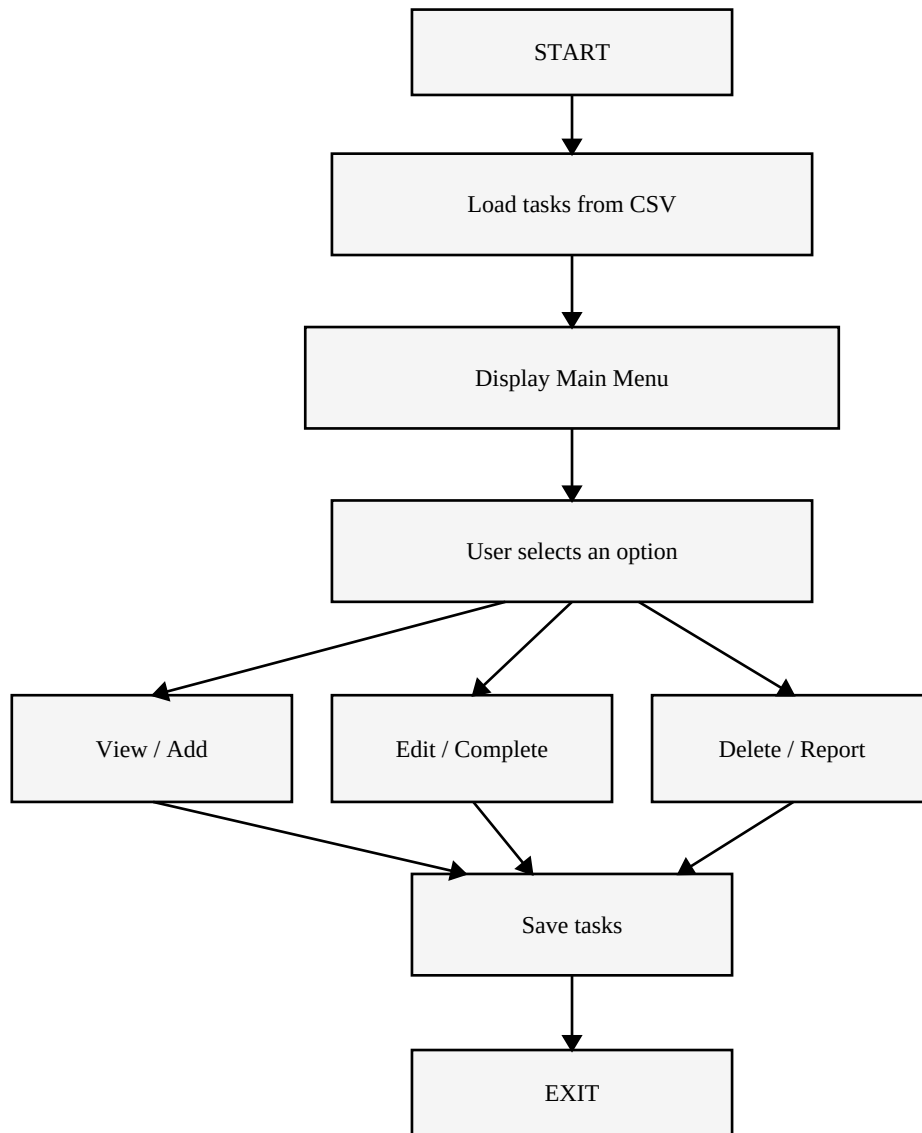


Figure 2: Main workflow of the application.

8. Use Case Diagram

The main actor is the User. The user can perform the task management and reporting operations shown below.

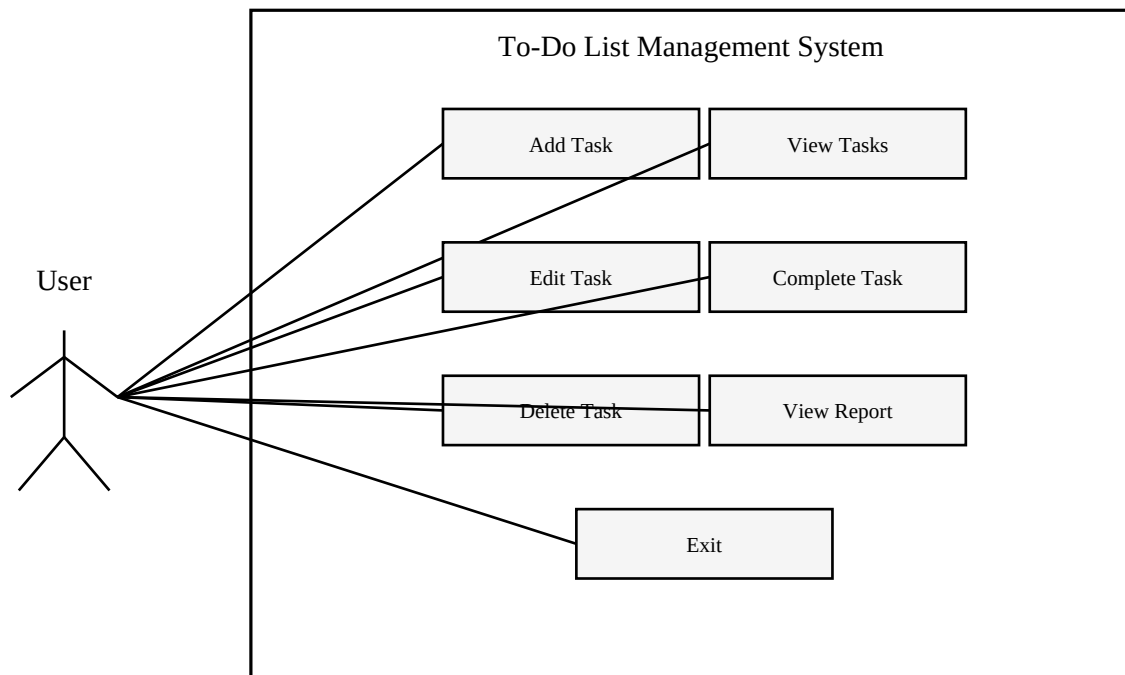


Figure 3: Use case diagram.

9. Component Diagram

Because the project uses functions and modules instead of classes, a component/module diagram is used.

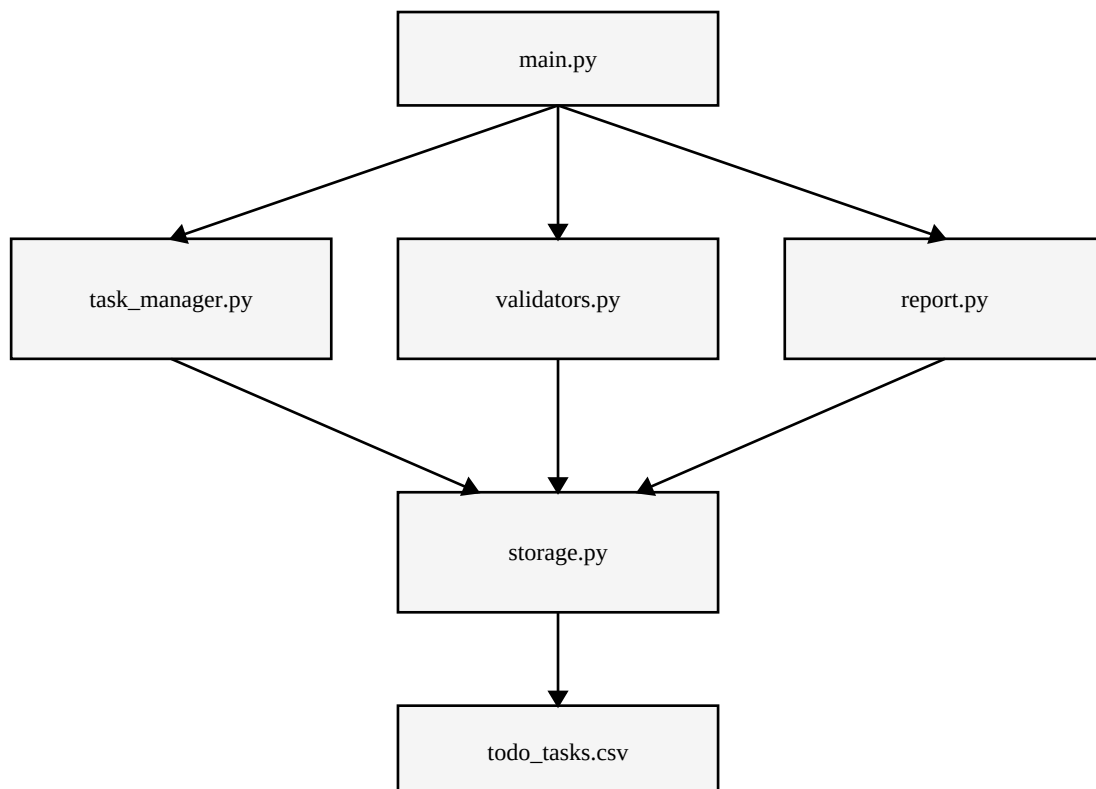


Figure 4: Python module/component structure.

10. Sequence Diagram

This example shows what happens when a user adds a task.

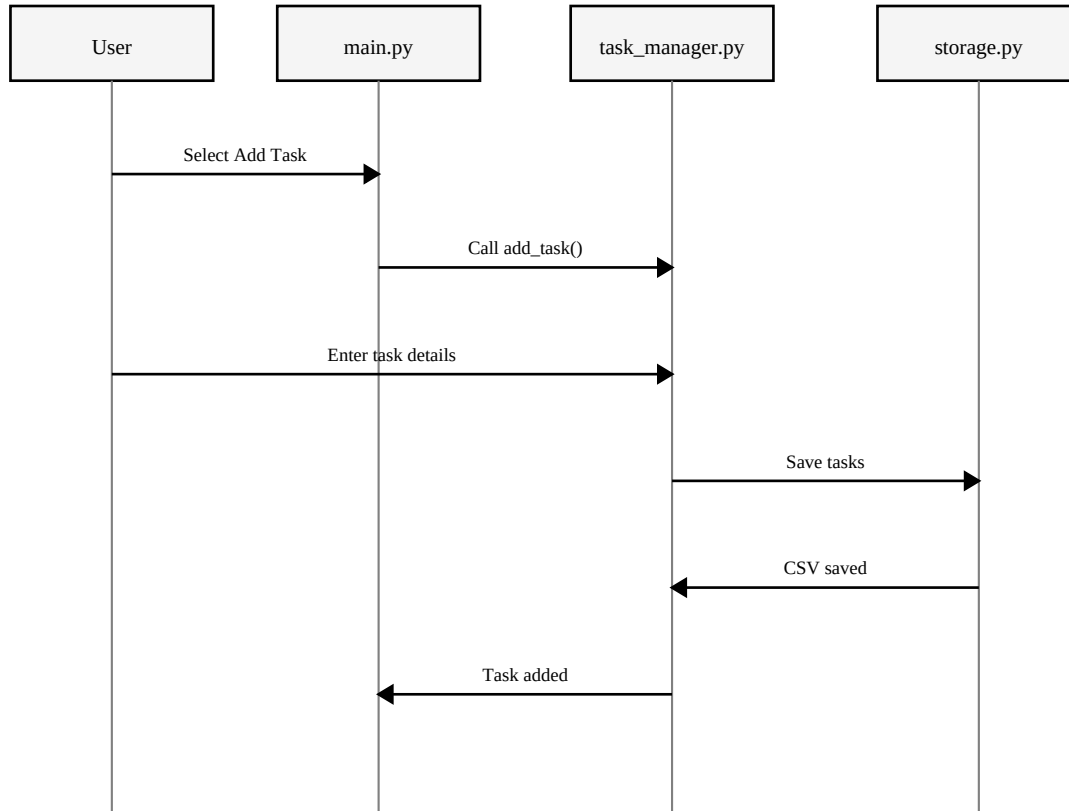


Figure 5: Sequence diagram for adding a task.

11. Storage Design

The project uses a CSV file named **todo_tasks.csv**. Each row represents one task. There are four fields.

Field	Example	Purpose
task	Study Python	Stores the task name.
category	Study	Groups the task.
priority	High	Shows importance.
status	Pending	Shows whether the task is completed.

Example CSV data:

Study Python, Study, High, Pending
Complete assignment, College, Medium, Pending
Read a book, Personal, Low, Completed

12. Design Decisions and Rationale

Python: Python was selected because its syntax is simple and it is suitable for learning programming concepts.

CSV storage: CSV was selected because the project is small and does not need a database.

Multiple files: The program is divided into modules so each part has a clear responsibility.

Command-line interface: A text menu keeps the project simple and focuses on programming concepts.

13. Implementation Details

main.py – Starts the program, displays the menu and calls the required functions.

task_manager.py – Contains functions for viewing, adding, editing, completing and deleting tasks.

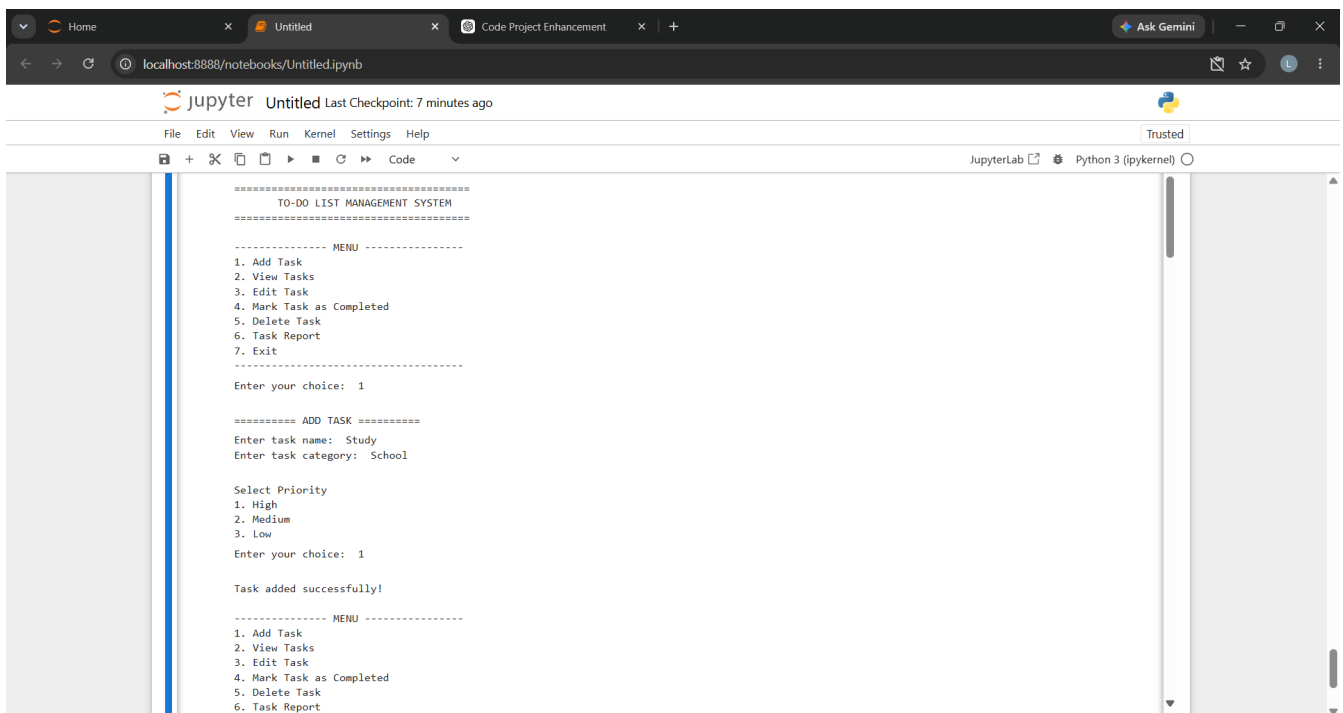
storage.py – Loads tasks from the CSV file and saves tasks to the CSV file.

validators.py – Checks priority values and task numbers and handles invalid numeric input.

report.py – Counts tasks and displays basic task statistics.

test_project.py – Contains simple checks for the task data structure.

14. Screenshots / Results



```
=====
TO-DO LIST MANAGEMENT SYSTEM
=====

----- MENU -----
1. Add Task
2. View Tasks
3. Edit Task
4. Mark Task as Completed
5. Delete Task
6. Task Report
7. Exit
-----
Enter your choice: 1

===== ADD TASK =====
Enter task name: Study
Enter task category: School

Select Priority
1. High
2. Medium
3. Low
Enter your choice: 1

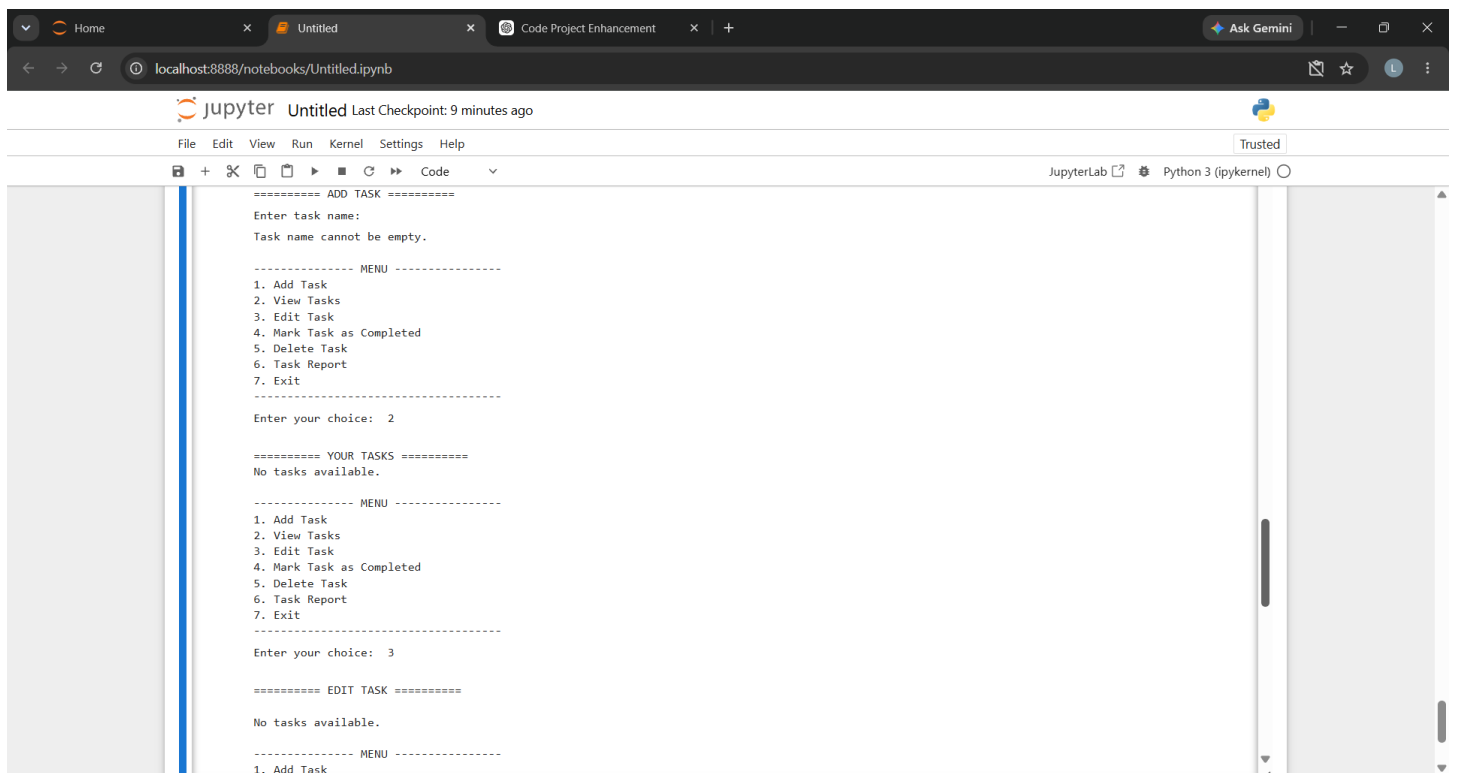
Task added successfully!

----- MENU -----
1. Add Task
2. View Tasks
3. Edit Task
4. Mark Task as Completed
5. Delete Task
6. Task Report
```

Figure 6: Adding a new task and selecting its priority.

14. Screenshots / Results (continued)

Actual program output captured from the working Python project.



```
===== ADD TASK =====
Enter task name:
Task name cannot be empty.

----- MENU -----
1. Add Task
2. View Tasks
3. Edit Task
4. Mark Task as Completed
5. Delete Task
6. Task Report
7. Exit

Enter your choice: 2

===== YOUR TASKS =====
No tasks available.

----- MENU -----
1. Add Task
2. View Tasks
3. Edit Task
4. Mark Task as Completed
5. Delete Task
6. Task Report
7. Exit

Enter your choice: 3

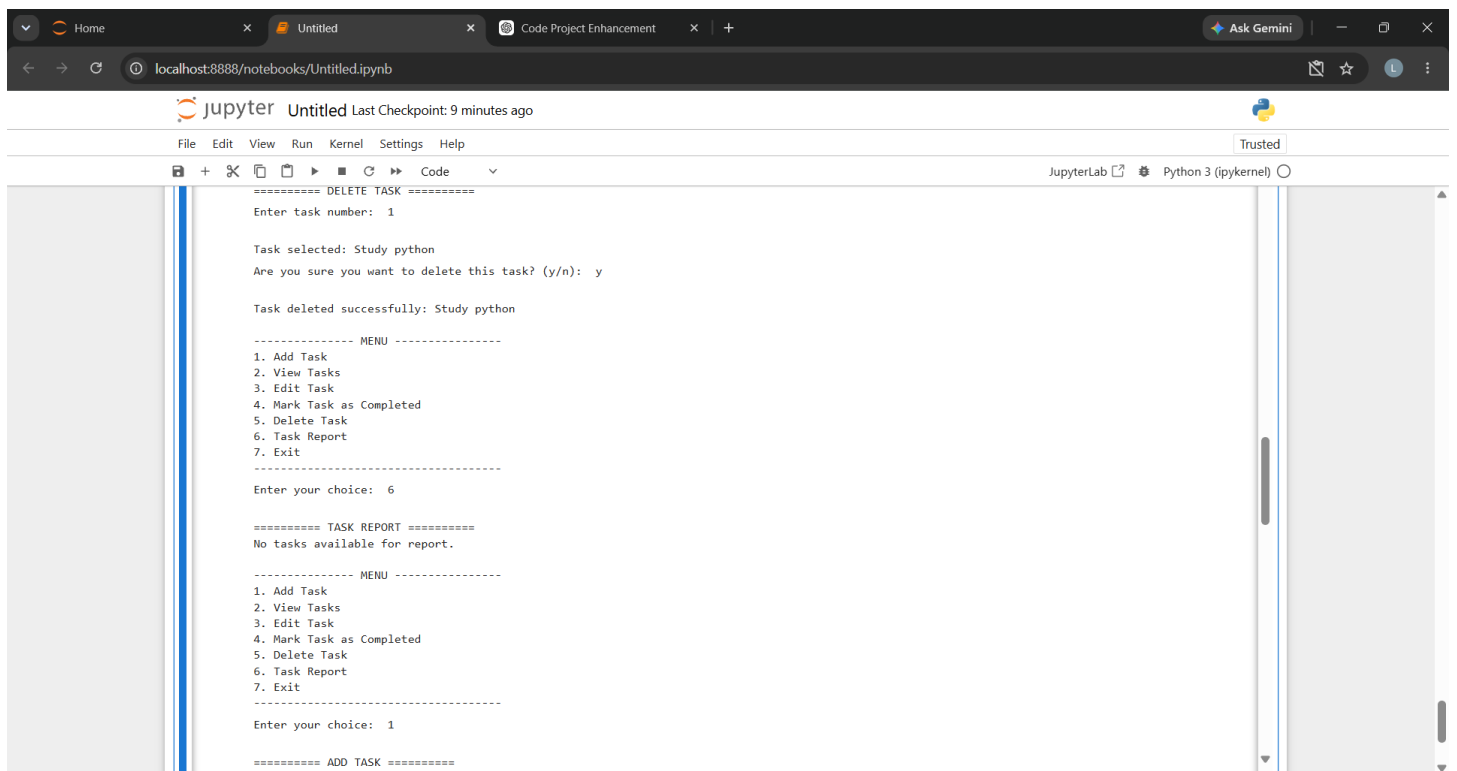
===== EDIT TASK =====
No tasks available.

----- MENU -----
1. Add Task
```

Figure 7: Viewing tasks and handling an empty task list.

14. Screenshots / Results (continued)

Actual program output captured from the working Python project.



```
===== DELETE TASK =====
Enter task number: 1

Task selected: Study python
Are you sure you want to delete this task? (y/n): y

Task deleted successfully: Study python

----- MENU -----
1. Add Task
2. View Tasks
3. Edit Task
4. Mark Task as Completed
5. Delete Task
6. Task Report
7. Exit
-----
Enter your choice: 6

===== TASK REPORT =====
No tasks available for report.

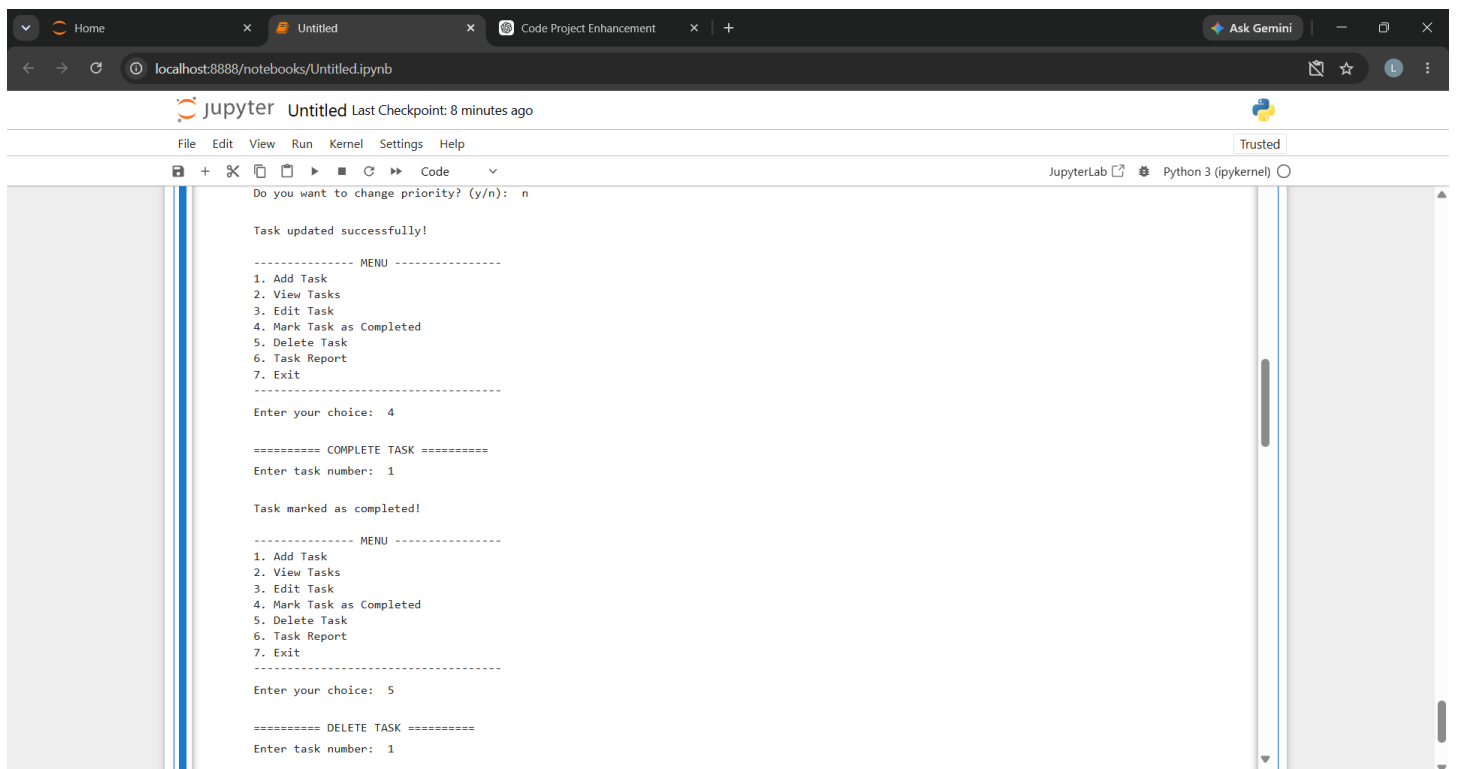
----- MENU -----
1. Add Task
2. View Tasks
3. Edit Task
4. Mark Task as Completed
5. Delete Task
6. Task Report
7. Exit
-----
Enter your choice: 1

===== ADD TASK =====
```

Figure 8: Deleting a task and viewing the task report.

14. Screenshots / Results (continued)

Actual program output captured from the working Python project.



```
Do you want to change priority? (y/n): n

Task updated successfully!

----- MENU -----
1. Add Task
2. View Tasks
3. Edit Task
4. Mark Task as Completed
5. Delete Task
6. Task Report
7. Exit
-----

Enter your choice: 4

===== COMPLETE TASK =====
Enter task number: 1

Task marked as completed!

----- MENU -----
1. Add Task
2. View Tasks
3. Edit Task
4. Mark Task as Completed
5. Delete Task
6. Task Report
7. Exit
-----

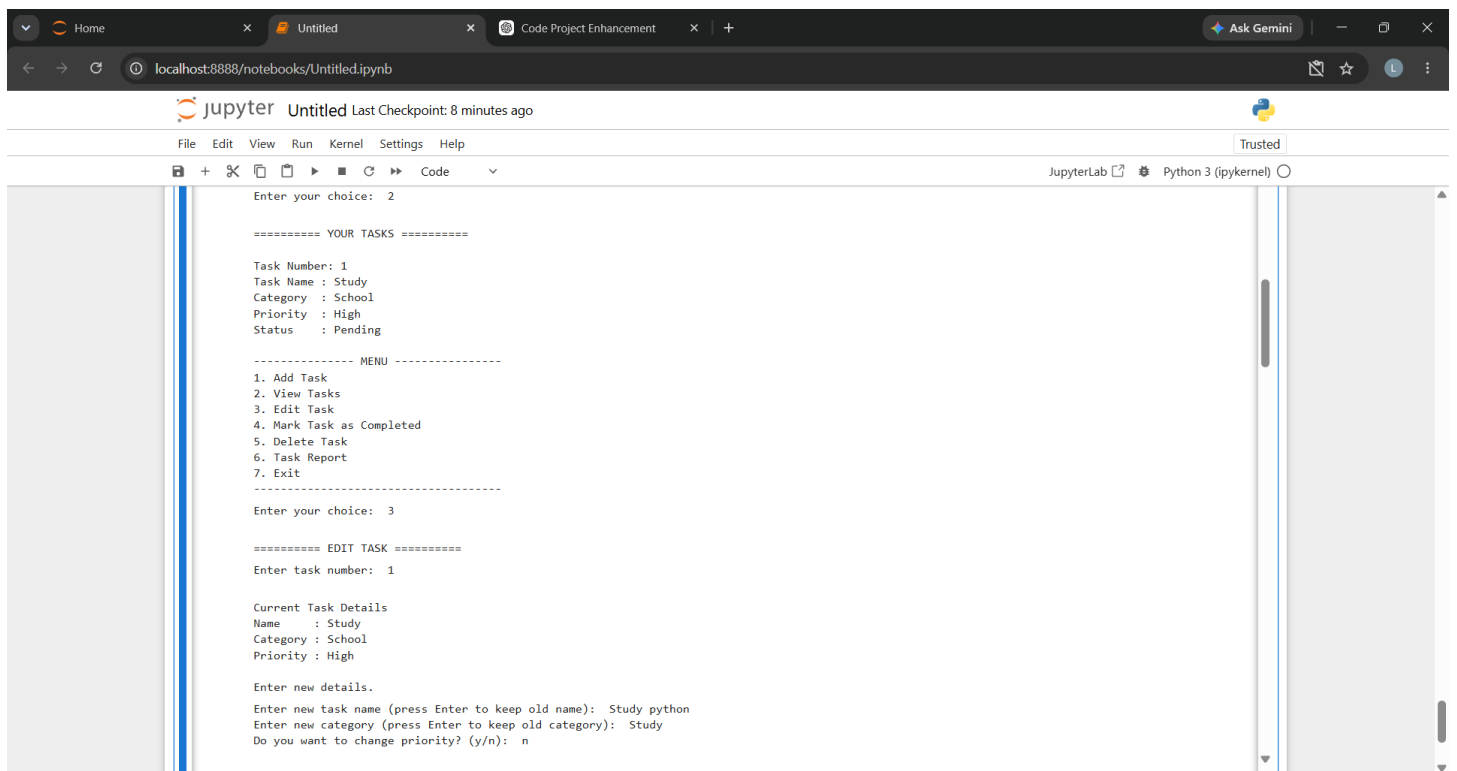
Enter your choice: 5

===== DELETE TASK =====
Enter task number: 1
```

Figure 9: Completing a task and returning to the main menu.

14. Screenshots / Results (continued)

Actual program output captured from the working Python project.



```
Enter your choice: 2

===== YOUR TASKS =====

Task Number: 1
Task Name : Study
Category  : School
Priority  : High
Status   : Pending

----- MENU -----
1. Add Task
2. View Tasks
3. Edit Task
4. Mark Task as Completed
5. Delete Task
6. Task Report
7. Exit
-----

Enter your choice: 3

===== EDIT TASK =====
Enter task number: 1

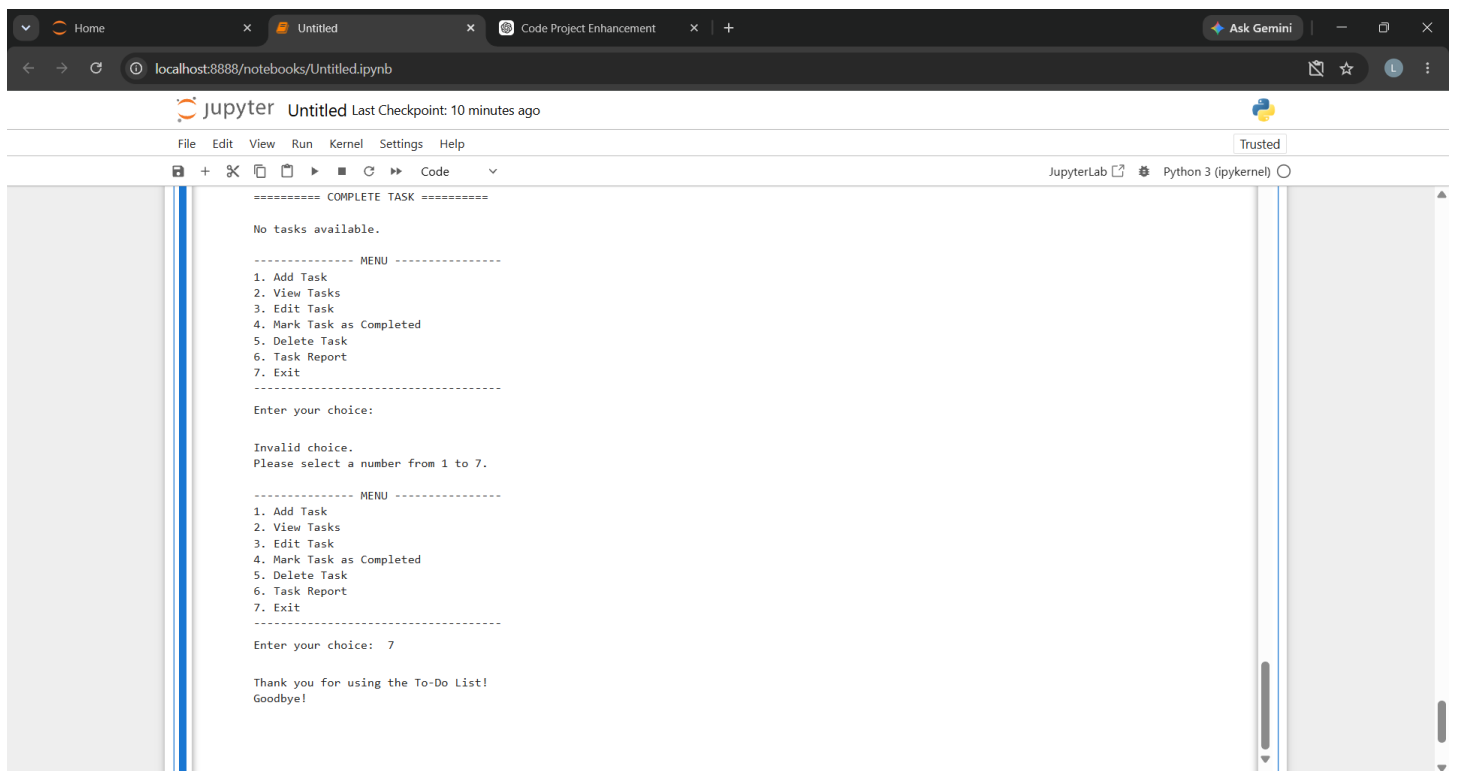
Current Task Details
Name      : Study
Category  : School
Priority  : High

Enter new details.
Enter new task name (press Enter to keep old name): Study python
Enter new category (press Enter to keep old category): Study
Do you want to change priority? (y/n): n
```

Figure 10: Viewing task details and editing a task.

14. Screenshots / Results (continued)

Actual program output captured from the working Python project.



```
===== COMPLETE TASK =====  
  
No tasks available.  
  
----- MENU -----  
1. Add Task  
2. View Tasks  
3. Edit Task  
4. Mark Task as Completed  
5. Delete Task  
6. Task Report  
7. Exit  
-----  
  
Enter your choice:  
  
Invalid choice.  
Please select a number from 1 to 7.  
  
----- MENU -----  
1. Add Task  
2. View Tasks  
3. Edit Task  
4. Mark Task as Completed  
5. Delete Task  
6. Task Report  
7. Exit  
-----  
  
Enter your choice: 7  
  
Thank you for using the To-Do List!  
Goodbye!
```

Figure 11: Invalid choice handling and exiting the application.

7. Exit

Figure 6: Sample main menu output. Take an actual screenshot of your program for the final report.

15. Testing Approach

Test Case	Input / Action	Expected Result
Add task	Enter a valid task	Task is added.
Empty task	Press Enter without a task	Error message is shown.
Invalid priority	Enter an invalid priority	User is asked again.
Complete task	Enter a valid task number	Status becomes Completed.
Invalid task number	Enter a number outside the list	Invalid task message is shown.
Non-numeric task number	Enter letters	Program asks for a number.
Delete task	Enter a valid task number	Task is removed.
View report	Select report option	Task statistics are displayed.

16. Challenges Faced

One challenge was understanding how to save information after the program closes. This was solved by using CSV file handling.

Another challenge was handling incorrect user input. Validation was added so that the program can respond to incorrect task numbers, priorities and other common inputs.

Dividing the project into different files was also a learning point. It helped in understanding how a larger program can be organized into smaller parts.

17. Learnings and Key Takeaways

This project helped me understand how Python can be used to solve a real-world problem. I learned how to use functions, lists, dictionaries, loops and conditional statements.

I also learned basic file handling by reading and writing CSV files. I learned the importance of input validation and error handling.

The project also introduced me to modular programming, where different parts of the program are kept in separate files.

18. Future Enhancements

- Add due dates for tasks.
- Add search and filtering.
- Add task reminders.
- Add a graphical user interface.
- Add user login and separate task lists.
- Use SQLite or another database for larger amounts of data.

- Add charts for task statistics.

19. Conclusion

The To-Do List Management System is a simple Python project that demonstrates how basic programming concepts can be combined to solve a practical problem. The system provides task management, validation, CSV storage and reporting features.

The project also demonstrates modular programming and basic testing. It provides a foundation that can be extended later with features such as reminders, a graphical interface and database storage.

20. References

1. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.
2. Python standard library documentation and course learning material.
3. Course notes and classroom material used during project development.

Declaration

I confirm that this report describes the To-Do List Management System project developed for the VITyarthi Build Your Own Project evaluation. The personal details on the cover page should be completed before submission.